

Deterministic Modelling

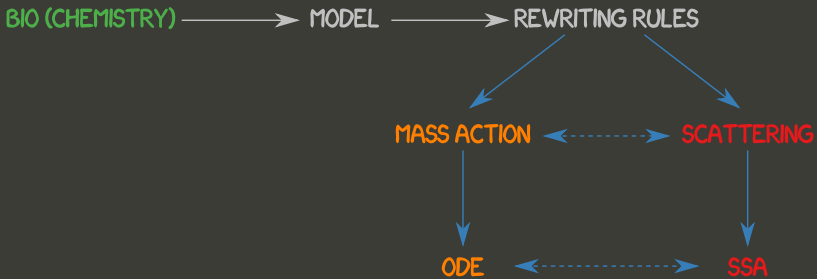
Dario Pescini¹

Università degli Studi di Milano-Bicocca

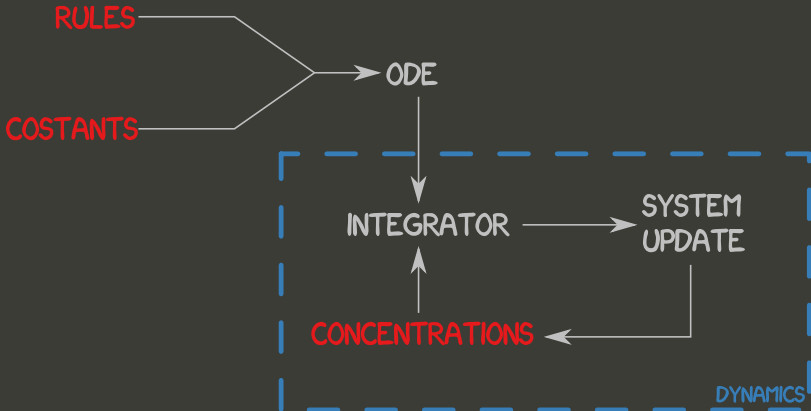
Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

Outline



Outline



ODE

Systems of **Ordinary Differential Equation** can be used whenever it is possible to establish a functional relationship between a function and its derivative.

- Given the system variation over time, ODEs reconstruct the system behaviour
- The Cauchy problem:**

$$\begin{cases} \frac{dy(t)}{dt} = f(y(t), t) \\ y(t=0) = y_0 \end{cases} \implies y(t) = ?$$



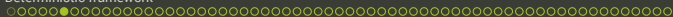
ODE: integrator

There exist many numerical approaches to solve ODEs, and the most simple is the **Euler's method**:

$$\frac{dy(t)}{dt} = f(y(t), t) \Rightarrow dy(t) = f(y(t), t)dt$$

by means of direct integration in $t_0, t_0 + dt$ we have:

$$\begin{aligned} y(t_1) &= y_1 = y(t=0) + f(y(t=0), t)dt \\ y(t_2) &= y_2 = y_1 + f(y_1, t)dt \\ y(t_3) &= y_3 = y_2 + f(y_2, t)dt \\ &\vdots \\ y(t_{n+1}) &= y_{n+1} = y_n + f(y_n, t)dt \end{aligned}$$



Euler's method: example

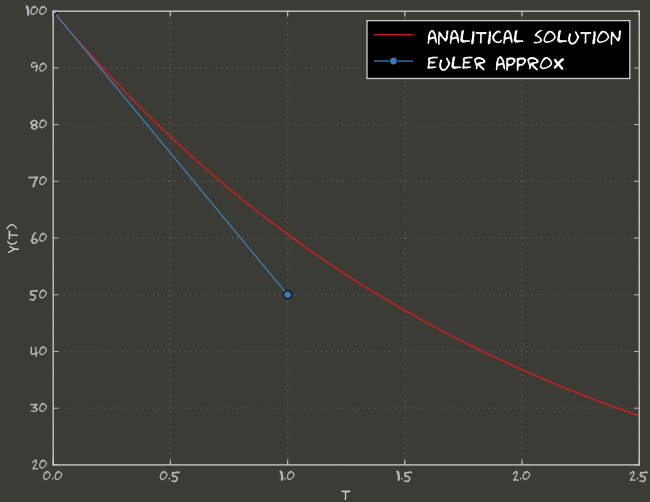
$$y_0 = 100$$

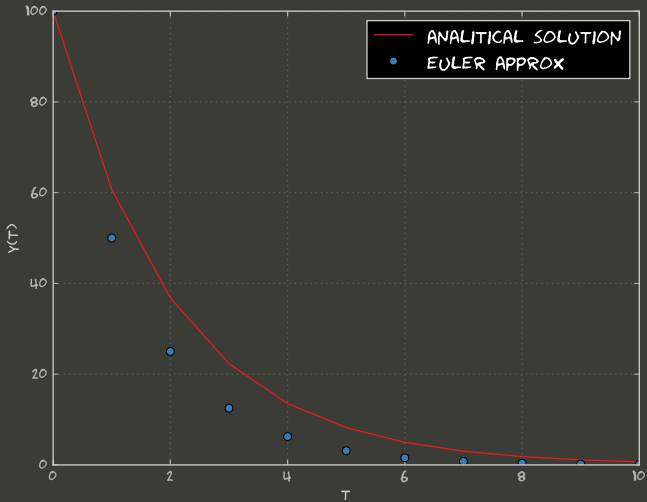
$$\begin{aligned} y_1 &= y_0 + f(t=0, y(t=0))\Delta t = \\ &= y_0 + (-k y_0) \Delta t = 100 - 0.5 \cdot 100 \cdot 0.1 = 95 \end{aligned}$$

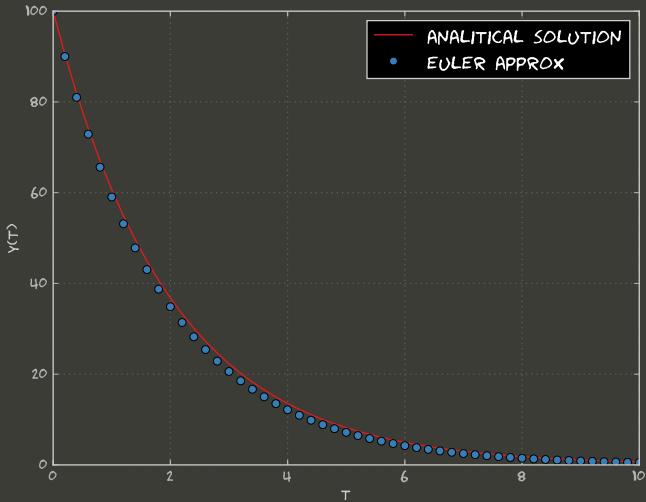
$$y_2 = y_1 + f(t_1, y_1)\Delta t = 95 - 0.5 \cdot 95 \cdot 0.1 = 90.25$$

$$y_3 = y_2 + f(t_2, y_2)\Delta t = 90.25 - 0.5 \cdot 90.25 \cdot 0.1 = 85.74$$

$$\vdots$$







ODE: integrator

It is possible to increase the accuracy: **4th order Runge Khutta method**

- The independent variable is increased by a constant amount h :
- The dependent variables are increased by an amount that is a **weighted average of 4 positions**:

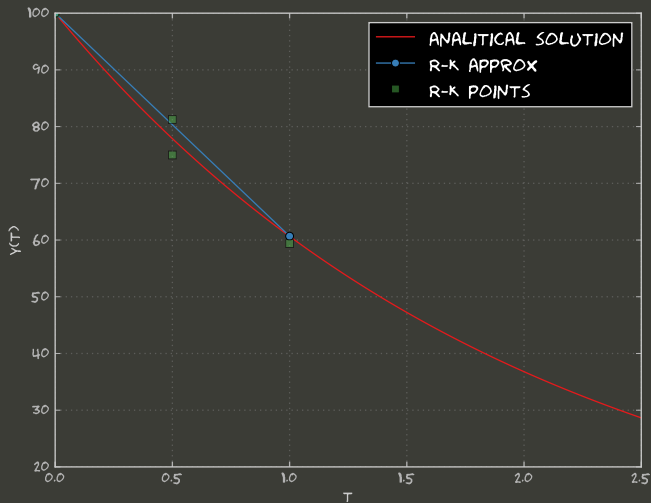
$$k_1 = f(t_n, y_n)$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + k_1 \frac{h}{2}\right)$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + k_2 \frac{h}{2}\right)$$

$$k_4 = f(t_n + h, y_n + k_3 h)$$

$$y_{n+1} = y_n + h \left(\frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} \right)$$



Runge-Kutta: example

$$k_1 = f(t_0, y_0) = -k y_0 = -0.5 \cdot 100 = -50$$

$$k_2 = f\left(t_0 + \frac{h}{2}, y_n + k_1 \frac{h}{2}\right) =$$

$$= f\left(100 - \frac{50}{2}\right) = -0.5 \cdot 75 = -37.5$$

$$k_3 = f\left(t_0 + \frac{h}{2}, y_n + k_2 \frac{h}{2}\right) =$$

$$= f\left(100 - \frac{37.5}{2}\right) = -0.5 \cdot 81.25 = -40.625$$

$$k_4 = f(t_0 + h, y_n + k_3 h)$$

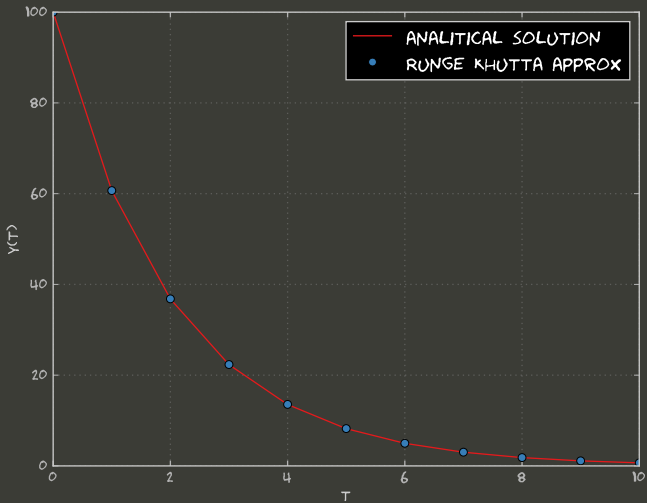
$$= f(100 - 40.625) = -29.6875$$

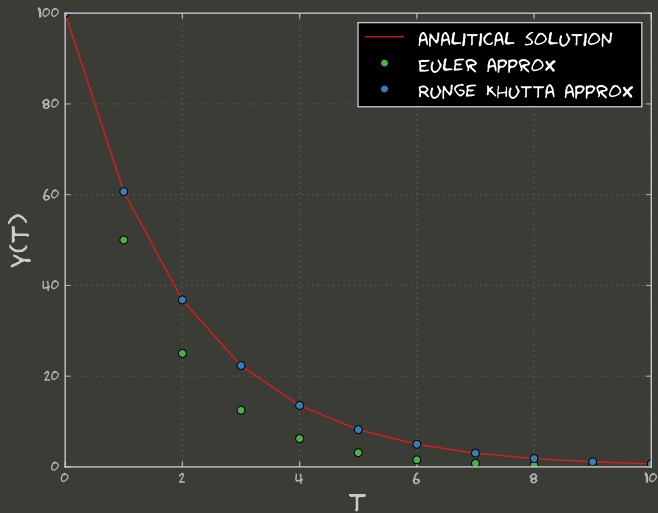
Runge-Kutta: example

$$\begin{aligned}
 y_1 &= y_0 + \Delta t \left(\frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} \right) = \\
 &= 100 + 1 \cdot \left(-\frac{50}{6} - \frac{37.5}{3} - \frac{40.625}{3} - \frac{29.685}{6} \right) = \\
 &= 60.6771
 \end{aligned}$$

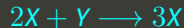
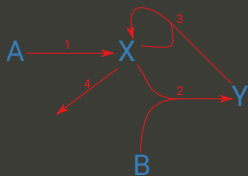
while

$$y(1) = 100 \cdot e^{-0.5 \cdot 1} = 60.6531$$





the BZ Reaction, the Brusselator & the Dynamics



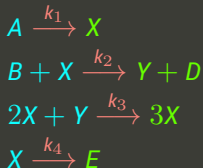
$$\begin{cases} \frac{\partial X}{\partial t} = k_1 A - k_2 B X + k_3 X^2 Y - k_4 X \\ \frac{\partial Y}{\partial t} = k_2 B X - k_3 X^2 Y \end{cases}$$

The hidden hypothesis

$$\begin{cases} \frac{\partial X}{\partial t} = k_1 A - k_2 B X + k_3 X^2 Y - k_4 X \\ \frac{\partial Y}{\partial t} = k_2 B X - k_3 X^2 Y \end{cases}$$

These differential equations have been derived according to the **mass-action law**

From rewriting rules to ODE: mass-action law



$$\frac{dA}{dt} = -k_1[A]$$

$$\frac{dB}{dt} = -k_2[B][X]$$

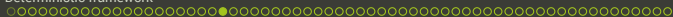
$$\frac{dX}{dt} = k_1[A] - k_2[B][X] + k_3[X]^2[Y] - k_4[X]$$

$$\frac{dY}{dt} = k_2[B][X] - k_3[X]^2[Y]$$

From rewriting rules to ODE: how to automate

$$\begin{array}{ccc}
 \alpha_{1,1}x_1 + \alpha_{1,2}x_2 + \cdots + \alpha_{1,N}x_N & \xrightarrow{k_1} & \beta_{1,1}x_1 + \beta_{1,2}x_2 + \cdots + \beta_{1,N}x_N \\
 \alpha_{2,1}x_1 + \alpha_{2,2}x_2 + \cdots + \alpha_{2,N}x_N & \xrightarrow{k_2} & \beta_{2,1}x_1 + \beta_{2,2}x_2 + \cdots + \beta_{2,N}x_N \\
 \vdots & & \vdots \\
 \alpha_{M,1}x_1 + \alpha_{M,2}x_2 + \cdots + \alpha_{M,N}x_N & \xrightarrow{k_M} & \beta_{M,1}x_1 + \beta_{M,2}x_2 + \cdots + \beta_{M,N}x_N
 \end{array}$$

$$\left\{ \begin{array}{l}
 \frac{dx_1}{dt} = k_1 \delta_{1,1} [x_1]^{\alpha_{1,1}} [x_2]^{\alpha_{1,2}} \dots [x_N]^{\alpha_{1,N}} + k_2 \delta_{2,1} [x_1]^{\alpha_{2,1}} [x_2]^{\alpha_{2,2}} \dots [x_N]^{\alpha_{2,N}} + \cdots + \\
 \quad k_M \delta_{M,1} [x_1]^{\alpha_{M,1}} [x_2]^{\alpha_{M,2}} \dots [x_N]^{\alpha_{M,N}} \\
 \frac{dx_2}{dt} = k_1 \delta_{1,2} [x_1]^{\alpha_{1,1}} [x_2]^{\alpha_{1,2}} \dots [x_N]^{\alpha_{1,N}} + k_2 \delta_{2,2} [x_1]^{\alpha_{2,1}} [x_2]^{\alpha_{2,2}} \dots [x_N]^{\alpha_{2,N}} + \cdots + \\
 \quad k_M \delta_{M,2} [x_1]^{\alpha_{M,1}} [x_2]^{\alpha_{M,2}} \dots [x_N]^{\alpha_{M,N}} \\
 \vdots \\
 \frac{dx_N}{dt} = k_1 \delta_{1,N} [x_1]^{\alpha_{1,1}} [x_2]^{\alpha_{1,2}} \dots [x_N]^{\alpha_{1,N}} + k_2 \delta_{2,N} [x_1]^{\alpha_{2,1}} [x_2]^{\alpha_{2,2}} \dots [x_N]^{\alpha_{2,N}} + \cdots + \\
 \quad k_M \delta_{M,N} [x_1]^{\alpha_{M,1}} [x_2]^{\alpha_{M,2}} \dots [x_N]^{\alpha_{M,N}}
 \end{array} \right.$$



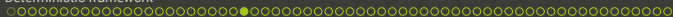
From rewriting rules to ODE: how to automate

$$\left\{ \begin{array}{l} \frac{dx_1}{dt} = k_1 \delta_{1,1} [x_1]^{\alpha_{1,1}} [x_2]^{\alpha_{1,2}} \dots [x_N]^{\alpha_{1,N}} + k_2 \delta_{2,1} [x_1]^{\alpha_{2,1}} [x_2]^{\alpha_{2,2}} \dots [x_N]^{\alpha_{2,N}} + \dots + \\ \quad k_M \delta_{M,1} [x_1]^{\alpha_{M,1}} [x_2]^{\alpha_{M,2}} \dots [x_N]^{\alpha_{M,N}} \\ \frac{dx_2}{dt} = k_1 \delta_{1,2} [x_1]^{\alpha_{1,1}} [x_2]^{\alpha_{1,2}} \dots [x_N]^{\alpha_{1,N}} + k_2 \delta_{2,2} [x_1]^{\alpha_{2,1}} [x_2]^{\alpha_{2,2}} \dots [x_N]^{\alpha_{2,N}} + \dots + \\ \quad k_M \delta_{M,2} [x_1]^{\alpha_{M,1}} [x_2]^{\alpha_{M,2}} \dots [x_N]^{\alpha_{M,N}} \\ \vdots \\ \frac{dx_N}{dt} = k_1 \delta_{1,N} [x_1]^{\alpha_{1,1}} [x_2]^{\alpha_{1,2}} \dots [x_N]^{\alpha_{1,N}} + k_2 \delta_{2,N} [x_1]^{\alpha_{2,1}} [x_2]^{\alpha_{2,2}} \dots [x_N]^{\alpha_{2,N}} + \dots + \\ \quad k_M \delta_{M,N} [x_1]^{\alpha_{M,1}} [x_2]^{\alpha_{M,2}} \dots [x_N]^{\alpha_{M,N}} \end{array} \right.$$

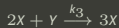
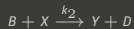
$$\left\{ \begin{array}{l} \frac{dx_1}{dt} = \sum_i^M k_i \delta_{i,1} \prod_p^N [x_p]^{\alpha_{i,p}} \\ \frac{dx_2}{dt} = \sum_i^M k_i \delta_{i,2} \prod_p^N [x_p]^{\alpha_{i,p}} \\ \vdots \\ \frac{dx_N}{dt} = \sum_i^M k_i \delta_{i,N} \prod_p^N [x_p]^{\alpha_{i,p}} \end{array} \right.$$

From rewriting rules to ODE: how to automate

$$\frac{dx_j}{dt} = \sum_i^M k_i \delta_{i,j} \prod_p^N [x_p]^{\alpha_{i,p}}$$

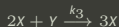
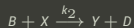


From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ \vdots \\ k_M \end{pmatrix}$$

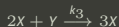
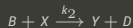
From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ \vdots \\ k_M \end{pmatrix}$$

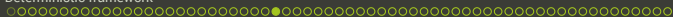
$$\frac{d[X]}{dt} =$$

From rewriting rules to ODE: Brusselator

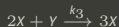
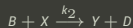


$$D = \begin{pmatrix} -1 & \textcolor{red}{1} & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} \textcolor{red}{k_1} \\ k_2 \\ \vdots \\ k_M \end{pmatrix}$$

$$\frac{d[X]}{dt} = \textcolor{red}{1} k_1$$



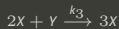
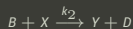
From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ \vdots \\ k_M \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1$$

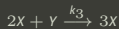
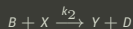
From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & \textcolor{red}{1} & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & \textcolor{blue}{0} & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} \textcolor{red}{k_1} \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

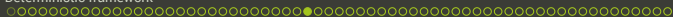
$$\frac{d[X]}{dt} = 1 \, k_1 x_1^1 x_2^0$$

From rewriting rules to ODE: Brusselator

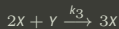
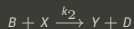


$$D = \begin{pmatrix} -1 & \textcolor{red}{1} & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & \textcolor{blue}{0} & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} \textcolor{red}{k_1} \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \ k_1 x_1^1 x_2^0 x_3^{\textcolor{blue}{0}}$$



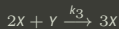
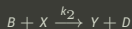
From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & \textcolor{red}{1} & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & \textcolor{green}{0} \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} \textcolor{red}{k_1} \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \ k_1 x_1^1 x_2^0 x_3^0 x_4^{\textcolor{green}{0}}$$

From rewriting rules to ODE: Brusselator

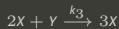
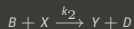


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0$$



From rewriting rules to ODE: Brusselator

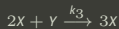
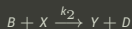


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1$$



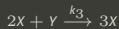
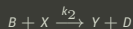
From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

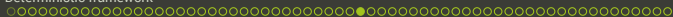
$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1$$

From rewriting rules to ODE: Brusselator

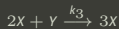
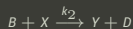


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0$$

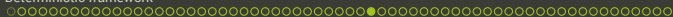


From rewriting rules to ODE: Brusselator

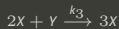
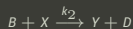


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & \textcolor{red}{1} & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ \textcolor{green}{0} & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ \textcolor{red}{k_3} \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0 + \textcolor{red}{k_3} x_1^0$$

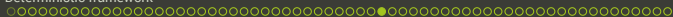


From rewriting rules to ODE: Brusselator

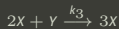
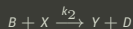


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & \textcolor{red}{1} & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & \textcolor{green}{2} & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ \textcolor{red}{k_3} \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^{\textcolor{green}{2}}$$

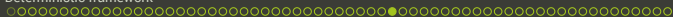


From rewriting rules to ODE: Brusselator

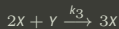
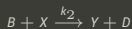


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & \textcolor{red}{1} & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & \textcolor{green}{0} & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ \textcolor{red}{k_3} \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^2 x_3^{\textcolor{green}{0}}$$

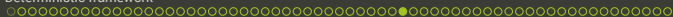


From rewriting rules to ODE: Brusselator

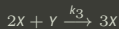
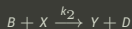


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & \textcolor{red}{1} & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & \textcolor{green}{1} \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ \textcolor{red}{k_3} \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^2 x_3^0 x_4^{\textcolor{green}{1}}$$

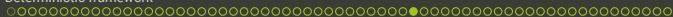


From rewriting rules to ODE: Brusselator

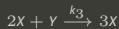
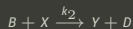


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^2 x_3^0 x_4^1 - k_4 x_1^0$$

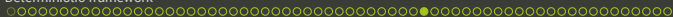


From rewriting rules to ODE: Brusselator

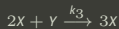
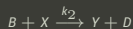


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^2 x_3^0 x_4^1 - k_4 x_1^0 x_2^1$$

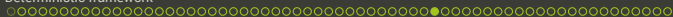


From rewriting rules to ODE: Brusselator

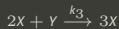
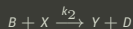


$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 \cdot k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 \cdot k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^2 x_3^0 x_4^1 - k_4 x_1^0 x_2^1 x_3^0$$



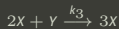
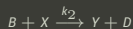
From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^2 x_3^0 x_4^1 - k_4 x_1^0 x_2^1 x_3^0 x_4^0$$

From rewriting rules to ODE: Brusselator



$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = k_1 x_1 - k_2 x_2 x_3 + k_3 x_2^2 x_4 - k_4 x_2$$

From rewriting rules to ODE: draft algorithm

$$\frac{dx_j}{dt} = \sum_i^M k_i \delta_{i,j} \prod_p^N [x_p]^{\alpha_{i,p}}$$

It is possible to recognize three cycles:

- running over matrix D one column at a time
- running over matrix D and vector $\vec{k}$ one row at a time
- running over matrix L one column at a time

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[X]}{dt} = 1 k_1 x_1^1 x_2^0 x_3^0 x_4^0 - 1 k_2 x_1^0 x_2^1 x_3^1 x_4^0 + k_3 x_1^0 x_2^2 x_3^0 x_4^1 - k_4 x_1^0 x_2^1 x_3^0 x_4^0$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 \, k_1 \, x_1^1 x_2^0 x_3^0 x_4^0$$

$$-1 \, k_2 \, x_1^0 x_2^1 x_3^1 x_4^0$$

$$+1 \, k_3 \, x_1^0 x_2^2 x_3^0 x_4^1$$

$$-1 \, k_4 \, x_1^0 x_2^1 x_3^0 x_4^0$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 \, k_1 \, x_1^1 x_2^0 x_3^0 x_4^0$$

$$-1 \, k_2 \, x_1^0 x_2^1 x_3^1 x_4^0$$

$$+1 \, k_3 \, x_1^0 x_2^2 x_3^0 x_4^1$$

$$-1 \, k_4 \, x_1^0 x_2^1 x_3^0 x_4^0$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 \, k_1 \, x_1^1 x_2^0 x_3^0 x_4^0$$

$$-1 \, k_2 \, x_1^0 x_2^1 x_3^1 x_4^0$$

$$+1 \, k_3 \, x_1^0 x_2^2 x_3^0 x_4^1$$

$$-1 \, k_4 \, x_1^0 x_2^1 x_3^0 x_4^0$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 \, k_1 \, x_1^1 x_2^0 x_3^0 x_4^0$$

$$-1 \, k_2 \, x_1^0 x_2^1 x_3^1 x_4^0$$

$$+1 \, k_3 \, x_1^0 x_2^2 x_3^0 x_4^1$$

$$-1 \, k_4 \, x_1^0 x_2^1 x_3^0 x_4^0$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 k_1 \quad x_1^1 x_2^0 x_3^0 x_4^0$$

$$-1 k_2 \quad x_1^0 x_2^1 x_3^1 x_4^0$$

$$+1 k_3 \quad x_1^0 x_2^2 x_3^0 x_4^1$$

$$-1 k_4 \quad x_1^0 x_2^1 x_3^0 x_4^0$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 \, k_1 \, x_1^1 x_2^0 x_3^0 x_4^0$$

$$-1 \, k_2 \, x_1^0 x_2^1 x_3^1 x_4^0$$

$$+1 \, k_3 \, x_1^0 x_2^2 x_3^0 x_4^1$$

$$-1 \, k_4 \, x_1^0 x_2^1 x_3^0 x_4^0$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 k_1 x_1^1 x_2^0 x_3^0 x_4^0$$

$$-1 k_2 x_1^0 x_2^1 x_3^1 x_4^0$$

$$+1 k_3 x_1^0 x_2^2 x_3^0 x_4^1$$

$$-1 k_4 x_1^0 x_2^1 x_3^0 x_4^0$$

`np.power(x, leftSide)`

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 \, k_1 \, x_1^1 \cdot x_2^0 \cdot x_3^0 \cdot x_4^0$$

$$-1 \, k_2 \, x_1^0 \cdot x_2^1 \cdot x_3^1 \cdot x_4^0$$

$$+1 \, k_3 \, x_1^0 \cdot x_2^2 \cdot x_3^0 \cdot x_4^1$$

$$-1 \, k_4 \, x_1^0 \cdot x_2^1 \cdot x_3^0 \cdot x_4^0$$

`np.power(x, leftSide)`

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 k_1 \quad x_1^1 \cdot x_2^0 \cdot x_3^0 \cdot x_4^0$$

$$-1 k_2 \quad x_1^0 \cdot x_2^1 \cdot x_3^1 \cdot x_4^0$$

$$+1 k_3 \quad x_1^0 \cdot x_2^2 \cdot x_3^0 \cdot x_4^1$$

$$-1 k_4 \quad x_1^0 \cdot x_2^1 \cdot x_3^0 \cdot x_4^0$$

```
np.product(np.power(x, leftSide),
axis=1)
```

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \vec{k} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 k_1 \quad \tau_1$$

$$-1 k_2 \quad \tau_2$$

$$+1 k_3 \quad \tau_3$$

$$-1 k_4 \quad \tau_4$$

```
np.product(np.power(x, leftSide), axis=1)
```

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \vec{k} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 k_1 \quad \tau_{2,1}$$

$$-1 k_2 \quad \tau_{2,2}$$

$$+1 k_3 \quad \tau_{2,3}$$

$$-1 k_4 \quad \tau_{2,4}$$

```
np.product(np.power(x, leftSide),
axis=1)
```


Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \vec{k} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 \, k_1 \quad \tau_{2,1}$$

$$-1 \, k_2 \quad \tau_{2,2}$$

$$+1 \, k_3 \quad \tau_{2,3}$$

$$-1 \, k_4 \quad \tau_{2,4}$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \vec{k} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$+1 k_1 \quad \tau_{2,1}$$

$$-1 k_2 \quad \tau_{2,2}$$

$$+1 k_3 \quad \tau_{2,3}$$

$$-1 k_4 \quad \tau_{2,4}$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[A]}{dt} = -1k_1[A]$$

$$\frac{d[X]}{dt} = +1k_1[A] - 1k_2[B][X] + 1k_3[X]^2[Y] - 1k_4[X]$$

$$\frac{d[B]}{dt} = -1k_2[B][X]$$

$$\frac{d[Y]}{dt} = +1k_2[B][X] - 1k_3[X]^2[Y]$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \vec{k} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$D^T * \vec{k} \quad \text{component-wise}$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \vec{k} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$D^T * \vec{k} \quad \text{component-wise}$$

$$D.T * K$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

$$\frac{d[A]}{dt} = -1k_1[A]$$

$$\frac{d[X]}{dt} = +1k_1[A] - 1k_2[B][X] + 1k_3[X]^2[Y] - 1k_4[X]$$

$$\frac{d[B]}{dt} = -1k_2[B][X]$$

$$\frac{d[Y]}{dt} = +1k_2[B][X] - 1k_3[X]^2[Y]$$

Automatic generation of ODE: exploiting numPy

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 \end{pmatrix} L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{\vec{k}} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{pmatrix}$$

```
(D.T * K).dot(tau)
```

```
def myCauchy(x, t):
    terms = np.product(np.power(x, leftSide), axis=1)
    teta = delta.T * aConstants
    dx = teta.dot(terms)
    for spec in dictFeedSpecs:
        dx[spec] = 0.0
    return dx
```


the Brusselator Dynamics: ODE

